

Building Our Own AI Extraction Pipeline: A Cost, Security, and Readiness Brief

Timothy Allen

May 2026

The headline. We built a pipeline that turns customer-service call transcripts into structured, searchable records on a single AWS spot **g6e.xlarge** GPU instance running Nemotron-Nano-30B for **\$16.89 to extract the full 95,831-call corpus — 5.6× cheaper than Gemini 2.5 Flash Batch** (\$94) on the same workload. The system runs entirely inside our AWS account, so customer data never leaves our compliance perimeter, and auto-scales to \$0/hr when there's no work. Quality is production- ready for 5 of 7 schema fields today; the remaining 2 need a one-day prompt edit before we'd dashboard against them.

The Decision in One Page

What we're solving. Customer-service call transcripts sit unstructured in storage. Analytics, QA scoring, and trend reporting all need them turned into structured records (intent, sentiment, agent behavior, outcome) so BI tools can query them.

Three options on the table:

	Hosted API (Gemini 2.5 Flash)	g6e.xlarge GPU (this project)	inf2.xlarge Neuron (smaller model)
Cost for 95k calls	\$94 (Batch) – \$189 (Standard)	\$17 (spot)	\$14 – \$28
Hourly rate	per-token billing	\$0.55 spot / \$1.86 on-dem	\$0.76 on-demand
Model	Gemini 2.5 Flash	Nemotron-Nano-30B (MoE)	Llama-3.1-8B (dense)
Data leaves account?	Yes	No	No
Compliance overhead	DPA + vendor risk review	Inherits VPC posture	Inherits VPC posture
Best when...	Realtime or sub-300-call jobs	Batch $\geq$ 750 calls + need 30B-class quality	Cost is the dominant constraint
Tradeoff	Data exits + per-token billing	Compute footprint	Lower-quality 8B model

Our pick: g6e.xlarge on AWS spot. Wins on cost today, wins on security posture, runs the higher-quality Nemotron-Nano-30B MoE model, and auto-scales to \$0/hr between batches. **inf2.xlarge is the alternative if cost becomes the dominant constraint** — per-hour it's 2.4× cheaper than g6e.xlarge (\$0.76 vs \$1.86 on-demand), and corpus cost lands in the same \$14–\$28

band as our spot pipeline. The tradeoff is the model: inf2.xlarge fits Llama-3.1-8B (dense, smaller), not Nemotron-Nano-30B (MoE). Cost-equivalent, lower-quality.

Caveat. For one-off jobs under 300 calls, hosted APIs win on cost and time-to-result. We solved this with a queue gate that holds small batches until ≥ 750 calls accumulate; if your usage is mostly small one-offs, this isn’t your tool.

1 What It Costs

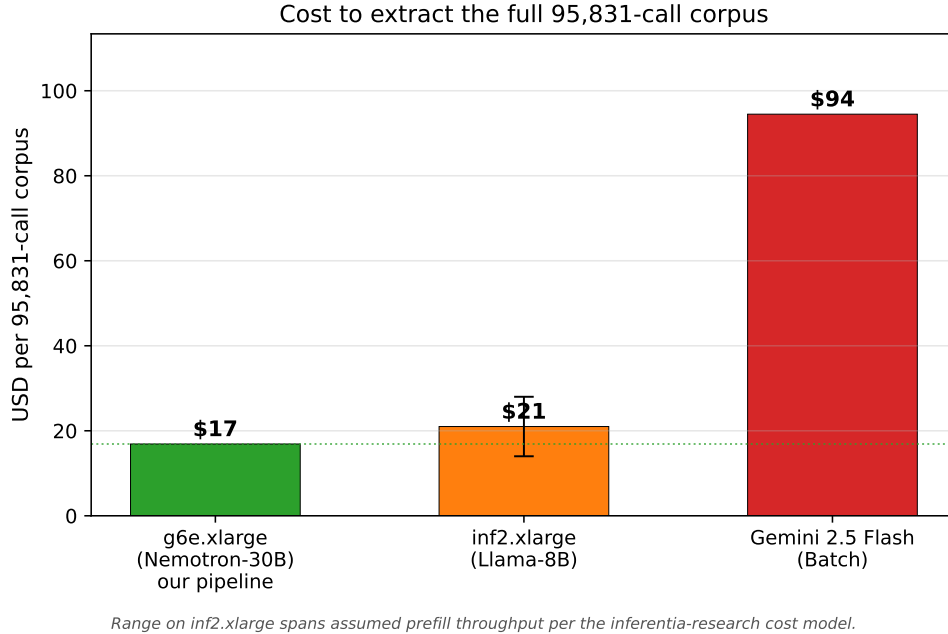


Figure 1: Cost to extract the full 95,831-call corpus. Our g6e.xlarge spot pipeline (green) lands at \$17 running Nemotron-Nano-30B. inf2.xlarge (orange, \$14–\$28 band) is cost-equivalent but constrained to a smaller Llama-3.1-8B model. Gemini 2.5 Flash Batch (red) is $5.6\times$ higher at \$94.

Pipeline		\$/hr	\$/call	\$/95 831 calls
g6e.xlarge — spot (our pipeline, Nemotron-Nano-30B)		\$0.55	\$0.000176	\$16.89
inf2.xlarge on-demand (Llama-3.1-8B) [†]		\$0.76	\$0.00015–\$0.00029	\$14–\$28
Gemini 2.5 Flash — Batch		—	\$0.000986	\$94.49

[†] Llama-3.1-8B BF16, TP=1, prefix-cache hit on system prompt; modeled at 1200–2400 prefill tok/s per the *inferentia-research* sibling project’s cost analysis.

What drives the cost difference. Hosted APIs charge per token, which adds up linearly with volume. Self-hosted, we pay only for compute time on a GPU instance we own for a few hours; spot pricing on AWS GPUs (\$0.55/hr vs. \$1.86/hr on-demand) cuts another 70%. The break-even is around 300 calls per batch — below that, hosted APIs are cheaper because our boot-up overhead dominates.

Why inf2.xlarge isn't our default. The per-hour rate (\$0.76) is $2.4\times$ cheaper than g6e.xlarge on-demand — but Nemotron-Nano-30B's Mamba layers and FP8 MoE kernels don't fit on Inferentia2, so the cost-effective deployment runs Llama-3.1-8B instead. That's a smaller, dense model and a meaningful quality step-down on this schema. The numbers above (cost, throughput) come from a paired modeling exercise in the `inferentia-research` sibling project, which dimensioned both instance pricing and Llama-3.1-8B prefill/decode rates against the same workload shape. The headline: *inf2.xlarge gets you roughly the same dollars per corpus, with a lower-quality model.*

The minimum-payload gate. Operationally we hold incoming batches until ≥ 750 calls accumulate, then process the whole queue in one spin-up. This pins us to the right side of the break-even curve regardless of incoming job shape. Trade-off: a small batch may wait minutes-to-hours for the queue to fill. Tunable knob, not a hard floor.

2 Why Self-Hosting Wins on Security

The dollar comparison understates the difference. The bigger decision driver, especially for regulated industries, is what happens to customer data.

Call transcripts containing names, addresses, DOBs, partial SSNs, and payment information **never leave our AWS account** when self-hosted. With a hosted API, every byte crosses the boundary into a third-party processor.

What changes when data stays local.

- **No Data Processing Agreement, no vendor risk review.** A hosted API path requires both, plus (in regulated sectors) a contractual subprocessor disclosure to end customers. Self-hosting sidesteps all three.
- **SOC 2 / HIPAA / PCI scope shrinks.** Inference happens inside our already-audited VPC and inherits the existing posture. With a hosted API, every prompt becomes a regulated data flow that has to be inventoried, retained, and audited.
- **Air-gapped option for the highest-regulation tenants.** Defense, healthcare, and financial-services back office often can't deploy hosted APIs at any price.
- **Data residency.** Self-hosting in a chosen AWS region trivially satisfies in-region requirements (EU, Canada, public-sector regions). Hosted APIs route through fixed regions that may not match.
- **Network cost.** Every byte stays on AWS's internal network — zero egress charges. Smaller than the compliance lever, but real for high-volume or large-payload workloads.

3 What We Built

An operator drops a JSONL file of call transcripts into an S3 inbox. The system detects it, scales up a single g6e.xlarge instance within 3–4 minutes, processes the batch, and shuts itself down — \$0/hr idle, ~\$15/month standing cost for persistent storage of model weights and Docker layers. The operator queries the resulting Iceberg table the same way they'd query any other warehouse table. Prompt and schema edits propagate to the running worker within five minutes via a cron-pulled config refresh — no instance replacement, no redeploy.

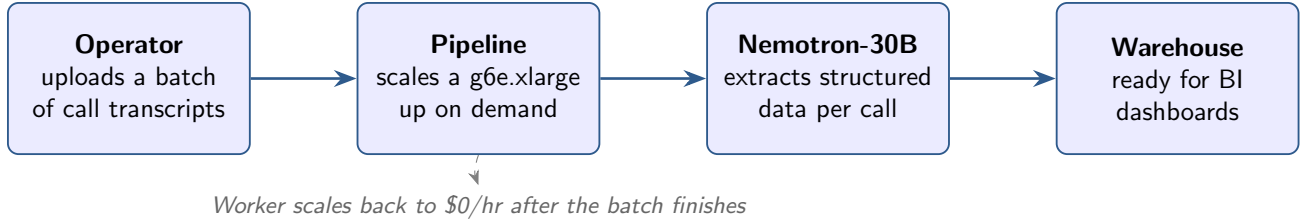


Figure 2: End-to-end flow. Operators see four business steps; the underlying AWS primitives (S3, SQS, ASG, Lambda, EBS, S3 Tables/Iceberg) are abstracted behind the pipeline.

4 What the Data Looks Like

We ran two extraction passes on the same paid AWS account against a public 92k-call dataset, writing results to the same Iceberg table.

Operational. 500 of 500 transcripts extracted successfully, 0 API errors. Median per-call latency 17 s; 95th percentile 20 s. The pipeline sustains 52 calls/min warm and projects to ~30 hours of compute for the full 95,831-call corpus.

Quality. We re-ran the same 200 calls on a freshly bootstrapped instance to check that (a) the infrastructure produces consistent results, and (b) the model is doing the same thing twice on the same input.

Field	Same value	Per-call agreement
outcome.resolution_status	190 / 199	95.5%
customer.satisfaction_score	180 / 199	90.5%
outcome.complexity_score	176 / 199	88.4%
agent.empathy_score	172 / 199	86.4%
agent.professionalism_score	170 / 199	85.4%
agent.knowledge_score	157 / 199	78.9%
customer.intent_primary	106 / 199	53.3%

Per-call agreement: percentage of calls where both runs produced the same value.

The good news (5 fields). `resolution_status` agrees 95% of the time on identical inputs — production-grade for any dashboard. The four 1–5 score fields (satisfaction, complexity, empathy, professionalism) agree 85–90%; where they disagree they disagree by exactly one point with no systematic bias. **Ready to scale today.**

The work to do (2 fields). `intent_primary` agrees only 53% — not because the model is wrong, but because it picks different surface forms (“compare auto insurance rates” vs. “compare_rates” vs. “request_quote”) for the same logical intent. Three boolean event flags fire at implausible rates because the prompt doesn’t define them tightly. **The fix is a one-day prompt edit** (anchor the intent enum to 15 values; add definitions and exemplars to the booleans).

The pass-2 run also exercised a re-architected configuration- management path (Ansible playbook with 5-minute cron reconcile, replacing the earlier hand-rolled bash). Per-call agreement was within sampling noise on every field — **the infrastructure migration introduced no quality regression**, the prerequisite for treating any of the cost numbers above as portable across deployments.

5 Customizing for an Enterprise Domain

A common follow-up: “*We have 30 industry-specific intents and custom compliance language. Can you fine-tune?*” The answer is almost always “not yet.”

Priority order, cheapest to most expensive: (1) prompt engineering + structured output → (2) RAG → (3) fine-tuning → (4) distillation, only as a last resort. Walk down the ladder only when the current rung’s data shape can’t satisfy the requirement.

Approach	Setup cost	Where it lives	When it wins
1. Prompt engineering + structured output	Hours	System prompt + Pydantic	Always start here. Covers ~70% of enterprise customization needs.
2. RAG over policy / schema docs	1–2 weeks	Vector store + retriever	The model needs facts it doesn’t have (compliance policies, taxonomies).
3. Fine-tune (LoRA or full)	\$100–\$500 (LoRA) to \$2k–\$20k (full) + weeks	Adapter or new checkpoint	Specialized vocabulary that can’t fit in-context, ≥1k labeled examples available.
4. Distillation (<i>last resort</i>)	Weeks–months + \$1k–\$50k teacher-inference	New small-model checkpoint	Sustained high volume where serving cost dominates everything else.

Why distillation is last. It produces a permanently cheaper inference footprint by training a small student model on a teacher model’s outputs, but the upfront cost (teacher inference + training + student deployment + ongoing quality regression) only pays off at sustained high volume. **The proof point** for this project: the two open quality issues (intent enum, boolean definitions) are both prompt fixes. We never needed fine-tuning, let alone distillation.

Concrete example: pinning the intent enum. Pass-1 produced 119 unique `intent_primary` surface forms. Replacing the free-form string with a Pydantic `Literal` type both anchors the model (the system prompt now lists the allowed values) and rejects off-list output post-hoc:

```
from typing import Literal
from pydantic import BaseModel, Field

class CustomerExtraction(BaseModel):
    intent_primary: Literal[
        "compare_auto_quotes",
        "purchase_auto_insurance",
        "file_claim",
        "policy_change",
        "billing_inquiry",
        # ... 9 more enterprise-specific intents ...
        "other",
    ] = Field(
        ...,
        description=(
            "Primary reason the customer called. "
            "Choose exactly one. Use 'other' if no value fits."
        ),
    )
```

The corresponding system-prompt addition is a single sentence: “*The intent_primary field must be one of the values listed in the schema. Do not invent new labels.*” Total cost of customization: one PR, ten lines of code, one prompt sentence. No retraining, no labeled data, no infrastructure change.

A frequently overlooked dimension is *who has the labels*. RAG and fine-tuning need either a curated document set or a labeled training subset; for many enterprises the cost of producing those (SME time, legal review) outstrips the cost of the model adaptation itself. Prompt engineering sidesteps that — the only “label” it requires is a written instruction.

6 Risks and How We Handle Them

Risk	Likelihood	Mitigation
AWS Spot interruption mid-batch	Medium (1–2% of hours)	Worker queries Iceberg for already-extracted call IDs and skips them on resume. Worst case: 5–10 min lost.
GPU quota constraint	Low	Pipeline holds at desired-capacity zero between batches; only burns quota during active jobs.
Schema regression on prompt edit	Medium	Two-pass re-extraction on the same 200-call sample catches it before scaling.
Vendor lock-in to AWS	Low	Iceberg open table format is portable to Snowflake, Databricks, BigQuery, or any S3-compatible store without re-extraction.

Not a risk: cost overruns (bounded by spot pricing), idle costs (\$0/hr by construction), or egress (\$0 in-region). The dominant operational risk is the schema-quality work — well- understood and reproducibly measurable.

7 Bottom Line

Across the four dimensions that drive this decision — **cost, compliance, operational shape, and data quality** — the self-hosted g6e.xlarge pipeline wins on every one. **\$17** to extract the full 95,831-call corpus, **5.6×** cheaper than Gemini 2.5 Flash Batch and **9×** cheaper than Standard. Customer PII stays inside our AWS account, eliminating a Data Processing Agreement and shrinking SOC 2 / HIPAA / PCI scope. **\$0/hr idle**, three-minute cold start, five-minute change propagation. **Five of seven** schema fields are dashboard-grade as-is; the remaining two close in a one-day prompt edit, not weeks of fine-tuning. The infrastructure migration between our two extraction passes introduced no quality regression, so the cost numbers travel across deployments.

Green-light the full 95,831-call extraction.

One day of schema-fix work (15-value intent enum + three tighter boolean definitions), then a ~30-hour spot run for ~\$17 of compute. Resume-by-run-id covers spot interruption; a Week-2 monitoring layer on the Iceberg table catches upstream drift. The only thing standing between us and the full corpus is a fifteen-line Pydantic edit.

Revisit only if our typical batch size drops below 300 calls per spin-up, an interactive use case enters the picture, or cost pressure makes the Llama-3.1-8B-on-inf2.xlarge quality tradeoff worth taking.